ASSIGNMENT 2 – TASK 2

CPU SCHEDULING – CLOUD VIDEO-ENCODING SERVICE

1. Introduction

CPU scheduling determines which ready process receives the processor and for how long. The given cloud video-encoding service handles short thumbnail jobs and long full-film jobs on shared servers. If a long job occupies the CPU first, short jobs may experience unacceptable delays. The scheduling policy must therefore balance responsiveness, throughput, fairness and processor utilisation.

2. Given Workload and Assumptions

The process data are taken directly from the assignment on page 13. All times are in abstract time units; the same workload is used for every algorithm.

Process	Arrival time	Burst time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Scheduling assumptions. FCFS and non-preemptive SJF select among jobs that have arrived; ties are broken by earlier arrival then process ID. SRTF is preemptive and selects the smallest remaining time; ties retain the earlier-arriving process. Round Robin uses a FIFO ready queue; processes arriving during a time slice are enqueued before the unfinished process is re-queued. The provided code uses quantum 3 by default.

Priority-data limitation. The assignment asks for Priority Scheduling but does not supply numeric priority values. To make the worked example reproducible, this report uses an explicitly illustrative convention: smaller number means higher priority, with P1=3, P2=1, P3=4 and P4=2. P1 runs first because it is the only available job at time 0; thereafter P2, P4 and P3 are selected. Replace these assumed priorities if your instructor provides a separate priority table.

3. Performance Measures

Completion time (CT) is the time at which a process finishes. Turnaround time (TAT) = CT – arrival time. Waiting time (WT) = TAT – burst time. Response time (RT) = first CPU start time – arrival time. Average values are the arithmetic mean over the four processes.

4. FCFS Scheduling

Non-preemptive; processes execute in arrival order.

Gantt chart: P1(0–8) → P2(8–12) → P3(12–21) → P4(21–26)

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	21	19	10	10
P4	26	23	18	18

Average TAT = 15.25; Average WT = 8.75; Average RT = 8.75.

4. SJF Scheduling

Non-preemptive; after P1, the shortest available burst is selected.

Gantt chart: P1(0–8) → P2(8–12) → P4(12–17) → P3(17–26)

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9

Average TAT = 14.25; Average WT = 7.75; Average RT = 7.75.

4. SRTF Scheduling

Preemptive; P2 pre-empts P1 at time 1, and P4 pre-empts P1 at time 5.

Gantt chart: P1(0–1) → P2(1–5) → P4(5–10) → P1(10–17) → P3(17–26)

Process	CT	TAT	WT	RT
P1	17	17	9	0
P2	5	4	0	0
P3	26	24	15	15
P4	10	7	2	2

Average TAT = 13.00; Average WT = 6.50; Average RT = 4.25.

4. Priority Scheduling

Non-preemptive priority using the illustrative priority values stated above.

Gantt chart: P1(0–8) → P2(8–12) → P4(12–17) → P3(17–26)

Process	CT	TAT	WT	RT
P1	8	8	0	0

Process	CT	TAT	WT	RT
P2	12	11	7	7
P3	26	24	15	15
P4	17	14	9	9

Average TAT = 14.25; Average WT = 7.75; Average RT = 7.75.

5. Round Robin Scheduling

Round Robin is preemptive. Each ready process receives at most one time quantum. If it is unfinished when the quantum expires, it is placed at the end of the ready queue. This provides each ready process with recurring CPU access and avoids indefinite starvation under a finite, continuously serviced queue.

5.1 Completed C implementation

The supplied TODO is completed by running the selected process for min(quantum, remaining burst), advancing time, enqueueing newly arrived processes in arrival order, and either re-queueing the unfinished process or recording its completion time. The full source file is supplied separately as round_robin.c.

```
#include <stdio.h>

#define N 4

int main(void)
{
    const char *name[N] = {"P1", "P2", "P3", "P4"};
    int arrival[N] = {0, 1, 2, 3};
    int burst[N] = {8, 4, 9, 5};
    int quantum = 3;

    int left[N], finish[N], queue[64], head = 0, tail = 0, added[N] = {0};
    for (int i = 0; i < N; i++) left[i] = burst[i];
    int time = 0, done = 0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while (done < N) {
        int p = queue[head++];

        /* Execute the selected process for one time quantum or its
           remaining burst time, whichever is smaller. */
        int run = (left[p] < quantum) ? left[p] : quantum;
        int start = time;
        time += run;
        left[p] -= run;
        printf(" %s(%d-%d)", name[p], start, time);

        /* Add all newly arrived processes in arrival order. */
        for (int i = 0; i < N; i++) {
            if (!added[i] && arrival[i] <= time) {
                queue[tail++] = i;
                added[i] = 1;
            }
        }

        /* Re-queue unfinished process; otherwise record completion. */
        if (left[p] > 0) {
            queue[tail++] = p;
```

```
            } else {
                finish[p] = time;
                done++;
            }
        }

        double tw = 0, tt = 0;
        printf("\nProcess TAT WT\n");
        for (int i = 0; i < N; i++) {
            int tat = finish[i] - arrival[i], wt = tat - burst[i];
            tt += tat;
            tw += wt;
            printf("%-7s %4d %3d\n", name[i], tat, wt);
        }
        printf("Average TAT = %.2f, average WT = %.2f\n", tt / N, tw / N);
        return 0;
    }
```

5.2 Quantum = 3: verification against the assignment

Gantt chart: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)

Process	CT	TAT	WT	RT
P1	23	23	15	0
P2	16	15	11	2
P3	26	24	15	4
P4	21	18	13	6

Average TAT = 20.00; Average WT = 13.50; Average RT = 3.00. The TAT and WT values match the expected output printed on page 13 of the assignment.

6. Round Robin Time-Quantum Investigation

The following results are calculated from the exact arrival and burst data. Context switches are counted as transitions from one process to a different process between consecutive CPU execution segments. The initial dispatch is not counted, and consecutive slices of the same process are not counted as a switch.

6.1 Quantum = 1

Gantt chart: P1(0-1) P2(1-2) P1(2-3) P3(3-4) P2(4-5) P4(5-6) P1(6-7) P3(7-8) P2(8-9) P4(9-10) P1(10-11) P3(11-12) P2(12-13) P4(13-14) P1(14-15) P3(15-16) P4(16-17) P1(17-18) P3(18-19) P4(19-20) P1(20-21) P3(21-22) P1(22-23) P3(23-24) P3(24-25) P3(25-26)

Process	CT	TAT	WT	RT
P1	23	23	15	0
P2	13	12	8	0
P3	26	24	15	1
P4	20	17	12	2

Average TAT = 19.00; Average WT = 12.50; Average RT = 0.75; Context switches = 23.

6.2 Quantum = 3

Gantt chart: P1(0-3) P2(3-6) P3(6-9) P4(9-12) P1(12-15) P2(15-16) P3(16-19) P4(19-21) P1(21-23) P3(23-26)

Process	CT	TAT	WT	RT
P1	23	23	15	0
P2	16	15	11	2
P3	26	24	15	4
P4	21	18	13	6

Average TAT = 20.00; Average WT = 13.50; Average RT = 3.00; Context switches = 9.

6.3 Quantum = 5

Gantt chart: P1(0-5) P2(5-9) P3(9-14) P4(14-19) P1(19-22) P3(22-26)

Process	CT	TAT	WT	RT
P1	22	22	14	0
P2	9	8	4	4
P3	26	24	15	7
P4	19	16	11	11

Average TAT = 17.50; Average WT = 11.00; Average RT = 5.50; Context switches = 5.

6.4 Quantum = 20

Gantt chart: P1(0-8) P2(8-12) P3(12-21) P4(21-26)

Process	CT	TAT	WT	RT
P1	8	8	0	0
P2	12	11	7	7
P3	21	19	10	10
P4	26	23	18	18

Average TAT = 15.25; Average WT = 8.75; Average RT = 8.75; Context switches = 3.

6.5 Comparison and interpretation

Quantum	Average TAT	Average WT	Average RT	Context switches
1	19.00	12.50	0.75	23
3	20.00	13.50	3.00	9
5	17.50	11.00	5.50	5
20	15.25	8.75	8.75	3

A small quantum improves the frequency with which waiting jobs receive CPU time, but causes more scheduling transitions and overhead. A large quantum reduces switching and approaches

FCFS behaviour; in this workload quantum 20 is longer than every remaining job when selected, so each process completes in one uninterrupted turn. Quantum 5 produces the lowest average turnaround and waiting times among the four tested quanta in this particular workload, while quantum 1 provides the lowest average response time. The best choice depends on the workload mix and the cost of a context switch.

7. Starvation and Ageing

Starvation occurs when a ready process waits indefinitely because other processes are repeatedly preferred. SJF can starve long jobs when short jobs keep arriving. Priority scheduling can starve low-priority jobs when higher-priority jobs continually enter the system.

Ageing reduces starvation by gradually increasing the priority of a process as its waiting time grows. For example, a waiting process may gain one priority level after each fixed waiting interval until it is eventually selected. Ageing should be bounded and applied consistently; it improves fairness without removing the need to monitor response-time targets.

8. Multilevel Feedback Queue (MLFQ)

An MLFQ scheduler maintains several ready queues with different priorities and time quanta. New or interactive jobs begin in a high-priority queue with a short quantum. A process that repeatedly consumes its full quantum is moved to a lower-priority queue with a longer quantum. Short thumbnail jobs are likely to finish quickly near the top, while long film-encoding jobs can continue in lower queues using longer slices, reducing repeated pre-emption overhead.

A practical MLFQ design periodically boosts waiting jobs or uses ageing to prevent low-queue starvation. Queue promotion, demotion and periodic boosts must be configured according to observed workload. MLFQ can provide good interactive response while retaining throughput for CPU-intensive encoding, but its behaviour depends on tuning and it is more complex than a single Round Robin queue.

9. Design Decision and Recommendation

For the supplied four-process example, SRTF has the lowest calculated average waiting time (6.50), average turnaround time (13.00) and average response time (4.25) among the listed non-RR algorithms. However, SRTF requires estimates of remaining CPU burst time and may starve long jobs under a continuous stream of short arrivals. The illustrative Priority result is dependent on the assumed priorities and should not be treated as a value specified by the assignment.

For the cloud video-encoding service, an MLFQ policy with ageing/periodic priority boosts is a defensible practical recommendation: short thumbnail jobs receive prompt service, while long encoding jobs are not permanently excluded and can run with longer quanta in lower queues. If the system requires a simpler implementation, Round Robin with a moderate quantum is a transparent alternative. The measured/observed workload and context-switch cost should guide

the final quantum.

10. Alternative Concept and Complexity Analysis: MLFQ

Under MLFQ, short-quantum queues serve new and interactive jobs first; jobs that repeatedly use their full allotment are demoted to queues with longer quanta. This lets short jobs complete with low initial waiting while amortising dispatch overhead for long jobs. Ageing or periodic boosts reduce starvation risk for long-running jobs.

With a linked-list or circular FIFO queue per priority level, enqueue and dequeue operations can be O(1) per queue. Selecting the highest non-empty queue is O(1) when a bitmap or direct queue index is maintained; otherwise it can be O(L), where L is the number of priority levels. SJF commonly requires O(n) scanning to select the shortest available job from n ready jobs (or O(log n) with a suitable priority heap). These are scheduling-decision costs, not total job-execution complexity.

For n processes, basic queue storage is O(n); the number of queue levels L is normally a small configured constant. SJF also stores O(n) ready-process metadata. MLFQ's principal trade-off is policy complexity and tuning, not asymptotic storage growth.

11. Conclusion

The scheduling calculations show that algorithm choice materially affects waiting, turnaround and response times. SRTF performs best on the supplied static four-job sample by the calculated averages, but its starvation risk and need for burst estimates limit its suitability as an unconditional production policy. For a service mixing short thumbnail jobs and long film encodes, MLFQ with ageing or periodic priority boosts offers a balanced design: fast initial service for short jobs and continued progress for long jobs. Round Robin remains a useful baseline, with quantum selection balancing responsiveness against context-switch overhead.

12. Execution and Reproducibility Notes

The C implementation is provided separately. Compile on Linux/Ubuntu/WSL with: gcc -std=c11 -Wall -Wextra round_robin.c -o round_robin. Run with: ./round_robin. The source defaults to quantum 3, matching the assignment's expected output. To test quantum 1 or 20, change the value of the quantum variable, recompile, and run again. Record your own machine/compiler details and screenshots in the README/evidence files; do not present another machine's environment as yours.